arplsbaseline Package Test Plan

Jessica Sam

Testing Instructions

This test plan should cover most, of the different cases that need to be tested. The baseline function is essentially the main function in the package and this requires the most testing, especially for input validation.

Package Structure

In my package I have a **baseline** function, which should create an object of class **Spectrum**.

Spectrum objects have callable elements and methods. Extractable elements are **baseline, wavenumber, original_signal** and **corrected_signal**, which should all be vectors of the same length. Methods that exist are **print, summary**, **plot**, **as.data.frame, baseline_gam** for objects of this class.

The package also contains sample spectra datasets, vignettes, as well as corresponding documentation for datasets, important functions and methods.

Testing Proper Package Functionality

These tests check that everything is present and do function as how it is expected. These are manual checks for the most important functionality for the users, whereas the unit tests, check everything automatically.

1. Vignettes

Calling the following:

```
browseVignettes(package = "arplsbaseline")
```

should correctly display links to the vignettes for the package, and can be opened as html pages.

2. Package Datasets

```
strawberry
ham
redwine
```

All three datasets should be readily available by calling their respective names.

3. Documentation

```
?strawberry
?baseline
```

Calling ? on the datasets or any one of the methods should bring up the respective documentation. In methods with examples, like the baseline documentation, clicking the examples embedded, should run as well.

4. Baseline function

```
spectrum <- baseline(strawberry)
```

Calling the baseline function on one of the datasets should return a Spectrum object and print output to the console about how long computation took and how many iterations.

5. Spectrum Methods

The created spectrum should be able to use the as.data.frame, plot, print, baseline_gam and summary methods.

```
plot(spectrum)
```

Plotting should show a visualisation with the baseline, original signal intensities and corrected spectrum.

```
as.data.frame(spectrum)
```

Dataframe coercion should work as expected and return a dataframe with all 4 columns of baseline, wavenumber, original_signal and corrected_signal.

```
print(spectrum)
```

Printing the spectrum should say what type of object it is, how many signals in the data used, and the extractable elements (baseline, wavenumber, original_signal, corrected_signal).

```
summary(spectrum)
```

Summary method should print output to the console with summary statistics with the corrected signals with respect to the computed baseline.

```
baseline_gam(spectrum)
model <- baseline_gam(spectrum, return_gam = TRUE)
```

The baseline_gam function should print summary output to the console about the GAM model it fit to the baseline as well as show a visualisation of the fitted model as well as the residuals. Setting the return_gam argument to TRUE, should also return an object of class “gam”.

Running Unit Tests

The package `testthat` should be installed in to the user's R packages, as well as this `arplsbaseline` package which is installed and loaded into the library.

To run all unit tests, the following command can be called:

```
testthat::test_package("arplsbaseline", "tap")
```

The results of this command should show 40 comments about all the expectations which have passed, and for what reason. An explanation of these tests are below.

Components Tested

The data used for the baseline testing are embedded into the test-baseline.R file.

The expectations numbered **1-19** are made regarding input validation of the **baseline function.**

Baseline Input Validation

1. Incorrect data format throws an error

   This test checks whether the function is properly throwing the error, **"Data must be in the form of a dataframe with 2 columns"**, whenever the function is given data that isn't a dataframe. There should be three expectations for this error, with NA, a vector [c(2,3)] and a single value [2] tested as inputs to the baseline function.

2. Dataframe has appropriate number of columns

   This test checks whether the function throws the error, **"Dataframe must only have 2 columns"** if the dataframe has more than 2 columns, this test uses a dataframe with 3 columns to check this.

3. NA values have been removed

   This test checks that a dataframe that contains a few NA values, but is otherwise reasonable will successfully compute a spectrum. It should be expected that both wavenumber and original_signal vectors that are extracted from the spectrum, no longer contain NA values and that the updated wavenumber and original_signal vectors are shorter than the wavenumber and signal vectors that are extracted from the input dataframe. There should be four expectations to pass.

4. The function throws an error if data is not numeric

   This test checks that if the function has a dataframe, but one of the columns is not numeric, that the error "**Both columns of dataframe must be numeric"** is thrown. A dataframe with a character element is used to test this condition.

5. The function throws an error if the data does not have enough elements

   This test checks that the error **"There must be at least 10 non-NA elements in each column"** is thrown, when each column of the dataframe doesn't have enough data to compute an inverse matrix on. This test uses a dataframe which has 9 elements in each column.

6. Lambda incorrectly entered, gives a message is given that the default value is used

   The test checks the message **"Lambda must be a single numeric value between 1 and 1e10, default lambda of 1e4 will now be used"** is thrown when lambda is entered incorrectly, where a vector c(1,2), or NA were used to test. This test has two expectations.

7. Function gives a message for incorrect masking arguments

   This test checks if different messages are returned if the function is given improper masking arguments. -The message **"At least one of the masking limits is not numeric, no masking will be used."** is checked by entering empty strings into each of the arguments. The message **"At least one of the masking limits is out of the range of the presented wavenumbers, no masking will be used."** is checked by setting the start_mask to a value 2, which is not a wavelength in the dataset, but the end_mask to 1500. The message **"Starting wavenumber to mask should not be larger than the end wavenumber, no masking will be used."** is checked by setting the start_mask to 1500 and the end_mask to 1400. The message **"One of the start or end limits for masking is missing, no masking will be used."** is checked by only providing one of the arguments. In total, there should be 6 expectations.

8. Function returns an error if not enough non-masked entries

   This test checks whether the error **"There must be at least 10 non-masked entries"** is thrown if the range of wavenumbers masked does not leave at least 10 valid wavenumbers to use for the baseline computation. This is tested with a dataframe with 18 elements, with 10 elements being masked.

Baseline Edge Cases

The expectations numbered **20-23** are made regarding edge cases of the **baseline function.**

9. Empty baseline function throws an error

   This test checks that the function throws an error, **"Data argument cannot be empty"**, when `baseline()` is called with no inputs.

10. Function throws an error if signals are all 0

    This test checks that the function throws an error, **"Signal intensity values cannot be all 0"**, when all the signal intensities are all 0.

11. Function throws an error if wavenumbers are not unique

    This test checks that the function throws an error, **"Wavenumbers must not contain duplicates",** if the wavenumbers include duplicates. In this, it was tested that all the wavenumbers are all the same value [0].

12. Function returns an error for Rcpp if dataset gets past input checks but still fails

    This test checks that in a random case that the function still fails after a dataset passes all the input validation checks, an error is thrown that tells the user essentially **"Rcpp failed"** and to use a different dataset.

Spectrum Components

The expectations numbered **24-40** are made regarding correct **Spectrum** functionality and structure, as well as input validation for baseline_gam arguments, given that the baseline function was successful. A spectrum made from the strawberry dataset within the package with a lambda of 1e7 was used for these tests.

13. The baseline function returns a Spectrum object

    This test confirms that the object that the baseline function returns is indeed of class Spectrum.

14. Each element from spectrum is a vector

    This test confirms that wavenumber, original_signal, baseline and corrected_signal are all vectors. There should be 4 expectations for this test.

15. Each vector is the same length

    This test confirms that wavenumber, original_spectrum, baseline and corrected_signal are all the same length. There should be 3 expectations for this test.

16. Spectrum can convert to dataframe

    This test confirms that the as.data.frame method on a Spectrum object, does in fact return a dataframe.

17. Summary methods give outputs

    This test confirms that the print and summary method on a Spectrum object, in this order, does in fact, print output to the console. There should be 2 expectations.

18. GAM method give outputs

    This test confirms that the baseline_gam method on a Spectrum object prints output to the console, and also returns an object of "gam" class if the return_gam argument is set to TRUE. There should be 2 expectations.

19. Incorrect GAM Method inputs give informative errors

    This test checks that the baseline_gam function returns the error **"full_summary and return_gam arguments can only be TRUE or FALSE"** if either one of the full_summary or return_gam values have been entered incorrectly. NA values and numeric values are tested as argument values. There should be 4 expectations.